Against the brief

Does the solution do what the take-home asked? Every requirement below is quoted from the kit's brief (python/README.md), with a verdict and where the evidence is. The design notes and grading material the kit mentions were not part of what we received, so this checks the brief as given.

In short: every stated requirement is met, most with measured evidence. The one open question is the brief's own warning about hidden test cases: the held-out run that answers it is built but not yet run.

What we expect

The brief says	Verdict	Evidence
"The receptionist only offers a sauna that can really be booked. If the API answers 422, the receptionist offered something wrong."	**Met**	Every offer and every write first passes `violations()`, a copy of the API's write rules, tested against the real API with hundreds of random bookings (`python/pirtis/agent/rules.py`, D-31). A booking is made only on a yes to the offer as read back (D-10, D-52). Every full gate has passed the kit's check that no API call fails
"It does not speak while the caller is still speaking. It does not leave a quiet caller waiting."	**Met**	The turn gate settles these with no model call: it waits on unfinished speech, a filler or "hang on", checks in after a silence and hangs up after a second one (`agent/turn.py`, D-04, D-05). Cases 02, 03, 26 and 40 pass every run
"It talks like a person on the phone. Short sentences. Numbers as words. One question at a time."	**Met, with one caveat**	Replies come from code templates; the `say` handler turns any digit left into words (D-17). Manual testing found one two-part question (fixed, D-96) and a tester's sense that replies feel rigid: fixed wording is the price of saying every fact correctly
"It passes the call to a human when needed. For example, when the caller is not a known customer; new customers cannot sign up by phone."	**Met**	A stranger is put through on the same turn, as are a request for a person, an angry caller who cuts in, two misunderstandings in a row and any API failure no branch handles (D-06, D-07, D-19). The transfer carries a note for the colleague
"You design the tools and the actions. Decide what your code should do and what the model should do."	**Met; our strongest point**	The model only records what the caller said, in one strict structured-output call; code checks it against the caller's own words and makes every decision, write and reply (D-02, D-08, D-54). The split is drawn in architecture/logic-flow.md; each decision, with what it replaced and the measurement behind it, is in SPEC §13 and the README's core technical decisions
"A new tool with two lines of comment on why is better than a prompt that tries to do the same."	**Met**	The rules mirror, the turn gate, the quote checks, the hedge rule and the name respelling are each code with a docstring saying why, instead of prompt rules. Where prompt rules were tried first, the decision log records that they didn't hold
"Every model call costs money and time … Most turns need one model call or none."	**Met**	0 or 1 model calls a turn; about $0.0004 a turn; p95 about 2 s; 15 of 138 gate runs need no model at all (evaluation.md)

What you may change, and what is fixed

The brief says	Verdict	Evidence
"Only two things are fixed on your side: the signatures of `decide` and `handlers`."	**Met**	Both unchanged (`agent/agent.py`, `agent/actions.py`)
"Three parts are ours. Do not change them": `pirtis/api`, `pirtis/llm`, `pirtis/harness` and `pirtis/cmd`	**Met**	Untouched. Our code lives in `pirtis/agent`, with `pirtis/ops` and `pirtis/playground` beside the kit, never imported by the agent. The kit's harness runs unchanged: `uv run harness`
"The model is always gpt-4o-mini, the same for every candidate."	**Met**	`decide()` calls gpt-4o-mini only. The off-path reviewer, which may use another model, runs after calls and never inside `decide()` (D-03, D-90)
Add action types, each with a handler	**Used**	`sms` (a written confirmation) and `note` (the turn's decision for tracing), each with its handler (`agent/actions.py`)

The tests, and the hidden ones

The brief says	Verdict	Evidence
"The model is not deterministic. A test case that passed once may fail next time; `--runs 3 --need 2` tells you more."	**Exceeded**	Our gate asks for 3 of 3, judge included, on all 46 cases; a 5-run reliability curve estimates what a 3-run gate misses (data-science.md §3, §4)
"We also have hidden test cases with the same rules but other callers, cars and dates. Make the rules work, not just the test cases."	**Open: the main risk**	For: hidden-style cases written to vary callers, cars and dates; fixes made as general rules, tested on wordings no case uses; a test that fails if the prompt quotes any case (D-88). Against: the 46 cases are also our development set, and one prompt leak showed how that can flatter a score. The 59 generated variants are kept as a held-out set (D-89); `make heldout` **has not been run yet**

Submitting

The brief says	Verdict	Evidence
"Be ready to explain every line, also the lines your tools wrote."	**Prepare**	The core is `python/pirtis/agent/`, about 2,300 lines. Everything else (the simulator, the console, the evaluation tools, the Azure deployment) needs a clear account of why it exists more than line-by-line fluency
"Plan for two to three hours."	**Far exceeded, by choice**	The work went well beyond the brief's budget: a caller simulator, an admin console, a measured evaluation and a live deployment. The agent itself is the answer to the task; the rest is how it was measured and shown working
"If you run out of time, write a short note about what you would do next."	**Met**	SPEC §12, "Known limits and next steps"

Open risks

1. **Hidden cases:** not measured out of sample until the held-out run.
2. **The judge is a model:** it has flipped on correct replies (about 1 in 43 on one case). Replies are worded to survive its known blind spots (D-51, D-72, D-87), but a hidden case's statements could still trip it.

3. **English only:** the agent's patterns and replies are English, and speech recognition of Lithuanian names depends on the recogniser. The kit's tests are text, so this affects the live app more than grading.

Before submitting

- [ ] Run `make heldout` and record the score as it comes, before tuning anything.
- [ ] Record the extractor eval and gate numbers for the deployed agent, `1c0de71d1184`, in the README and SPEC §7b.
- [ ] Freeze the agent: no further agent changes unless the held-out run shows a real bug.
- [ ] Walk through `python/pirtis/agent/` file by file, ready to explain each decision.